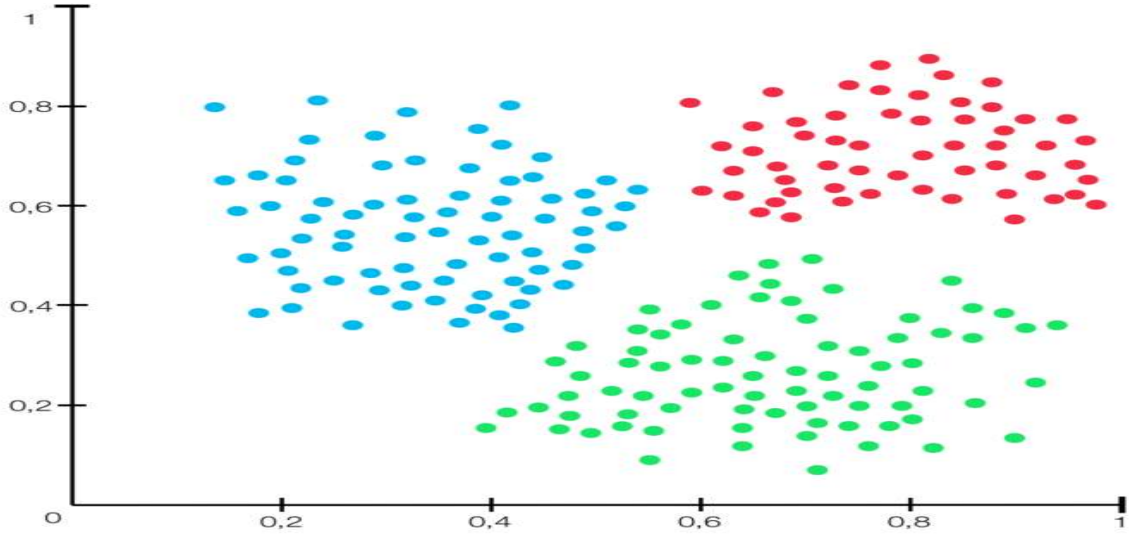
Unit - 4

1. Cluster Analysis:
2. Types of Data in Cluster Analysis
3. Major Clustering Methods
4. Partitioning Methods
5. Hierarchical methods
6. Density-Based Methods
7. Grid-Based Methods
8. Model-Based Clustering Methods
9. Clustering High-Dimensional Data
10. Constraint-Based Cluster Analysis
11. Outlier Analysis.



What is Cluster Analysis?

- Cluster Analysis is a **data mining technique** used to group similar objects together.
- **Similar to each other**
- **Different from objects in other clusters**

Real-World Examples

- Amazon groups customers based on buying behavior.
- Hospitals group patients based on symptoms.
- Schools group students based on performance.

Types of Data in Cluster Analysis

- In clustering, the **type of data** is very important because the **distance measure changes** based on data type.
- There are mainly **5 types of data** used in cluster analysis.

Interval-Scaled Data (Numeric Data)

- Numeric data where differences are meaningful, but there is **no true zero**.

Examples:

- Temperature (°C)
- IQ score
- Marks

Binary Data

- Data with only **two values** (0 or 1).

Example:

- Pass (1), Fail (0)
- Male (1), Female (0)
- Yes (1), No (0)

Nominal Data

- Categorical data with **no order**.

Examples:

- Color: Red, Blue, Green
- Department: CSE, ECE, IT
- Blood Group: A, B, AB, O

Ratio-Scaled Data

- Numeric data with a **true zero point**.

Examples:

- Height
- Weight
- Age
- Salary

Major Clustering Methods:

The clustering methods can be classified into following categories:

1. Kmeans
2. Partitioning Method
3. Hierarchical Method
4. Density-based Method
5. Grid-Based Method
6. Model-Based Method
7. Constraint-based Method

1. K-Means Clustering

•**Explanation**: It groups data into ‘K’ clusters based on similarity. Each cluster has a center, and data points are assigned to the nearest center.

•**Advantage**: Works well with large datasets and is easy to implement.

•**Example**: Grouping customers in an online store based on their shopping behavior to give personalized recommendations.

2. Partitioning Method

•**Explanation**: Divides data into ‘K’ clusters, where each data point belongs to one and only one cluster. K-Means is a common partitioning method.

•**Advantage**: Simple and efficient for small to medium datasets.

•**Example**: Organizing students in a class into study groups based on their scores.

3. Hierarchical Method

•**Explanation**: Builds a tree-like structure (called a dendrogram) by either merging smaller clusters or splitting bigger ones.

•**Advantage**: Can be used when the number of clusters is unknown.

•**Example**: Grouping species of animals based on their genetic similarities.

4. Density-Based Method

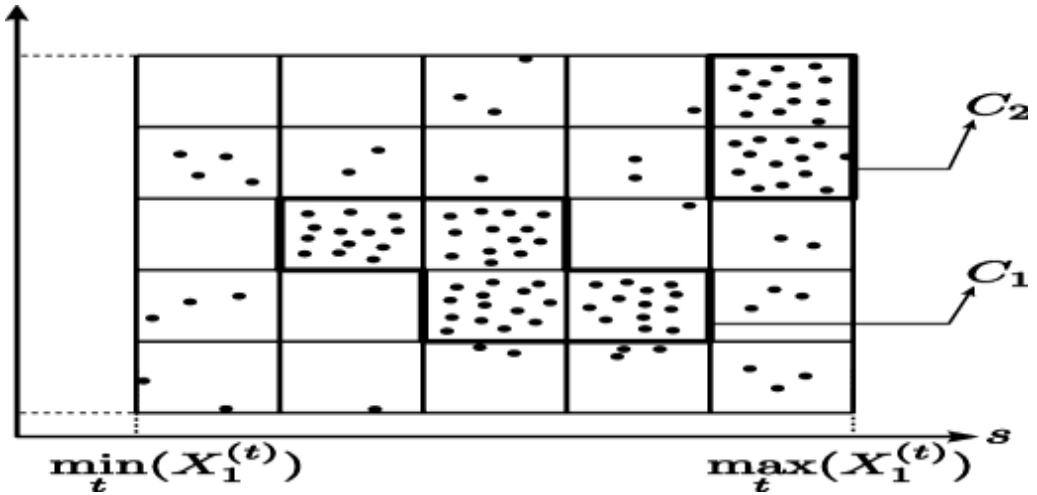
•**Explanation**: Finds clusters based on areas of high density and ignores noise (outliers).

•**Advantage**: Works well when clusters are of different shapes and sizes.

•**Example**: Detecting different regions of crowd density in a shopping mall using surveillance data.

5. Grid-Based Method

- **Explanation**: Divides the data space into grids and groups the dense grids into clusters.
- **Advantage**: Fast and efficient for large datasets.
- **Example**: Weather forecasting, where temperature data is divided into grids to identify climate patterns.



6. Model-Based Method

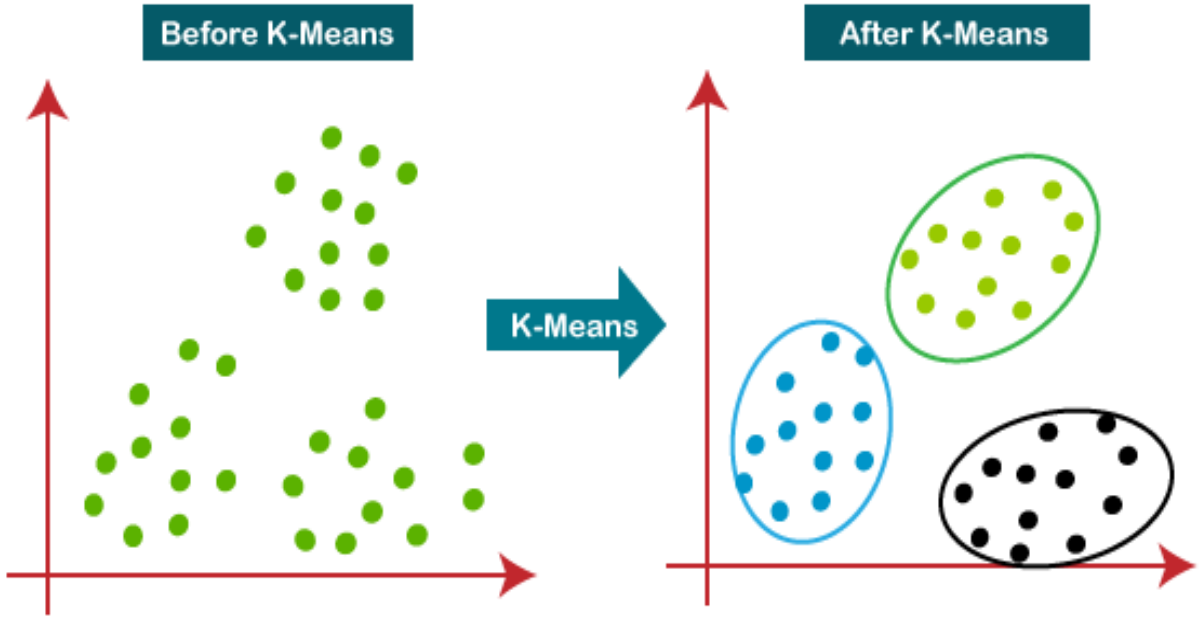
- **Explanation**: Assumes data follows a certain mathematical model (like Gaussian distribution) and assigns clusters accordingly.
- **Advantage**: Can handle noise and uncertainty well.
- **Example**: Identifying different types of handwritten digits in a digit recognition system.

7. Constraint-Based Method

- **Explanation**: Clusters are formed based on user-defined rules or constraints (like size, shape, or distance).
- **Advantage**: More control over clustering results based on specific needs.
- **Example**: Assigning delivery zones for a courier company while ensuring equal workload among delivery agents.

What is Partitioning Method?

- Partitioning method divides the dataset into **K number of clusters**.
- Each object belongs to **only one cluster**
- Number of clusters (K) must be given in advance



K means Algorithm

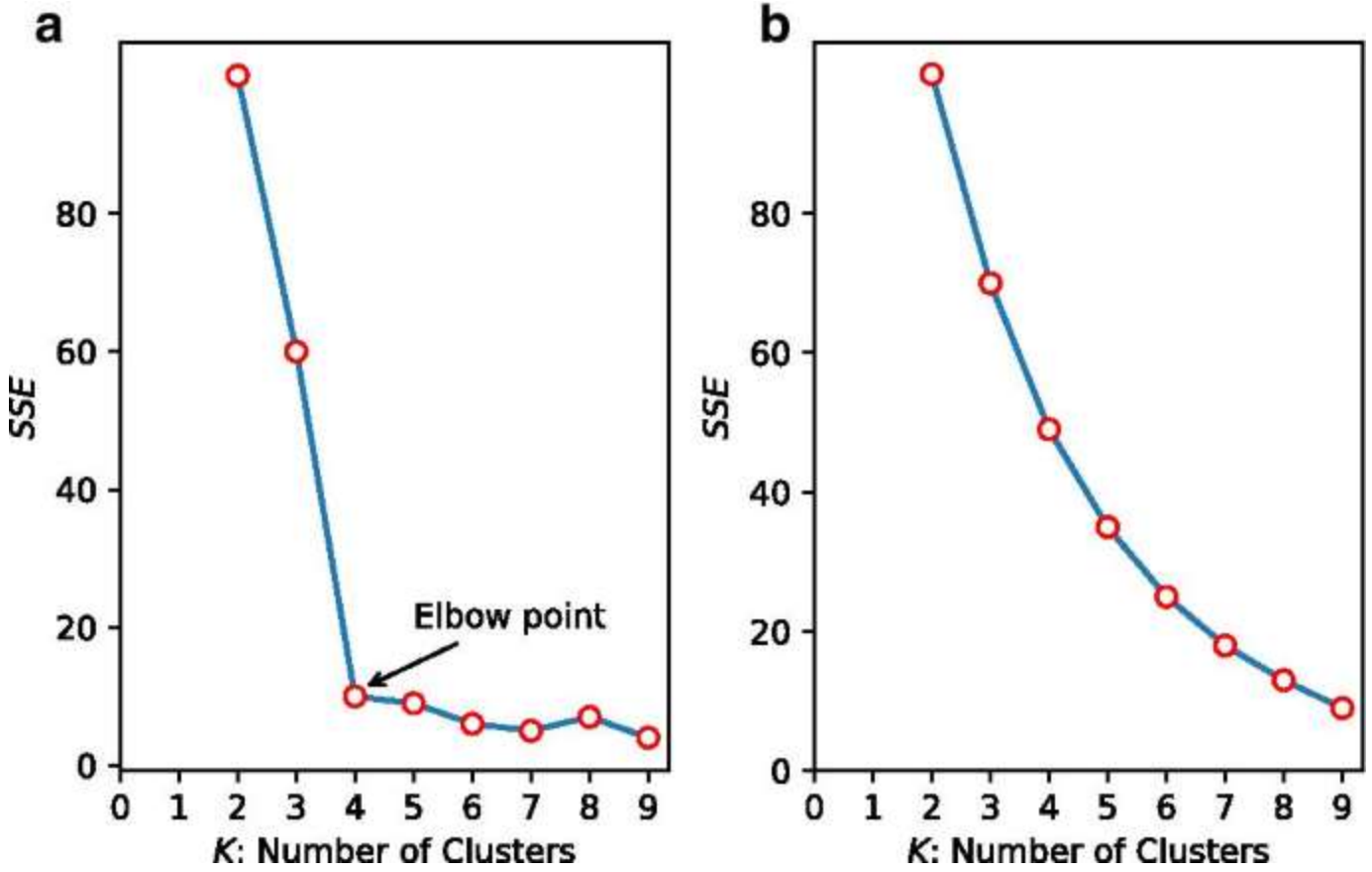
Step1: Randomly select the centroids

Step2:Euclidean distances with all data points

Step3:Re-calculate the centroids

Step4:repeat the 2 and 3 step until finding best centroids

Elbow method or plot



Example

K-Means Clustering Algorithm – Solved Example

- Use K Means clustering to cluster the following data into two groups.
- Data Points: { 2, 4, 10, 12, 3, 20, 30, 11, 25 }
- The distance function used is Euclidean distance.
- Initial cluster centroid are M1 = 4 and M2 = 11.

Initial Centroids:
M1: 4
M2: 11

Data Points	Distance to		Cluster	New Cluster
	M1	M2		
2	2	9		
4	0	7		
10	6	1		
12	8	1		
3	1	8		
20	16	9		
30	26	19		
11	7	0		
25	21	14		

$$d(x_2, x_1) = \sqrt{(x_2 - x_1)^2}$$

K-Means Clustering Algorithm – Solved Example

Current Centroids:
M1: 7
M2: 25

Final Cluster are:
C1= {2, 4, 10, 11, 12, 3}
C2= {20, 30, 25}

Data Points	Distance to		Cluster	New Cluster
	M1	M2		
2	5	23	C1	C1
4	3	21	C1	C1
10	3	15	C1	C1
12	5	13	C1	C1
3	4	22	C1	C1
20	13	5	C2	C2
30	23	5	C2	C2
11	4	14	C1	C1
25	18	0	C2	C2

$$d(x_2, x_1) = \sqrt{(x_2 - x_1)^2}$$

Time complexity

O(nkt)

Disadvantages

Not smater initialization of centroids

Real world example

- Custmer segmentation
- Image segmentation
- Netfilx
- Retail shops

Hierarchical Clustering Methods

- Hierarchical clustering is a type of unsupervised machine learning algorithm used to group objects into clusters based on their similarity.
- Unlike partition-based methods (such as k-means), hierarchical clustering does not require the number of clusters to be specified in advance.
- Instead, it creates a hierarchy of clusters that can be visualized in a tree-like structure called a **dendrogram**.
  There are two main types of hierarchical clustering:
  - **Agglomerative (Bottom-Up)**
  - **Divisive (Top-Down)**

Agglomerative Hierarchical Clustering (Bottom-Up)

- In **agglomerative clustering**, each object starts in its own cluster, and clusters are merged together step by step until only one cluster remains.
- The merging process is done based on a distance (or similarity) metric.

Steps:

1.Start with each object as a separate cluster (initially, there are as many clusters as objects).

2.Compute the distance between all pairs of clusters using a distance metric (e.g., Euclidean distance).

3.Merge the two closest clusters.

4.Repeat steps 2 and 3 until only one cluster remains (the root of the hierarchy).

- **Linkage Criteria:**
- **Single Linkage (Nearest Neighbor)**: The distance between two clusters is the distance between the closest pair of points (one in each cluster).
- **Complete Linkage (Furthest Neighbor)**: The distance between two clusters is the distance between the farthest pair of points (one in each cluster).
- **Average Linkage**: The distance between two clusters is the average distance between all pairs of points from the two clusters.
- **Ward's Method**: Minimize the total within-cluster variance by merging clusters.

- **Dendrogram:**
- A **dendrogram** is a tree-like diagram that records the merging process at each step.
- The height of the lines in the dendrogram represents the distance between clusters at each merging step.
- You can "cut" the dendrogram at a chosen height to obtain a desired number of clusters

2. Divisive Hierarchical Clustering (Top-Down)

- In **divisive clustering**, the process starts with all objects in a single cluster, and clusters are recursively split into smaller clusters.
- This approach is the opposite of agglomerative clustering.
- **Steps:**

1.Start with all objects in one cluster.

2.Recursively split the cluster into two or more clusters based on some criterion (e.g., distance or similarity).

3.Repeat until each object is in its own cluster or the stopping criterion is met (e.g., a desired number of clusters).

Divisive methods are less commonly used because they tend to be computationally more expensive compared to agglomerative methods.

- **Example of Hierarchical Clustering**
- **Given Data:**
- Let's consider a small dataset of points in a two-dimensional space:

$$\text{Data} = \begin{bmatrix} (2,3), \\ (3,3), \\ (6,5), \\ (8,8) \end{bmatrix}$$

Step-by-Step Agglomerative Clustering:

- **Initial Clusters**:
  - Start with each point as its own cluster: {(2,3)},{(3,3)},{(6,5)},{(8,8)}.

- **Distance Calculation**:
- Compute the distance between all pairs of clusters. For simplicity, let's use **Euclidean distance**.

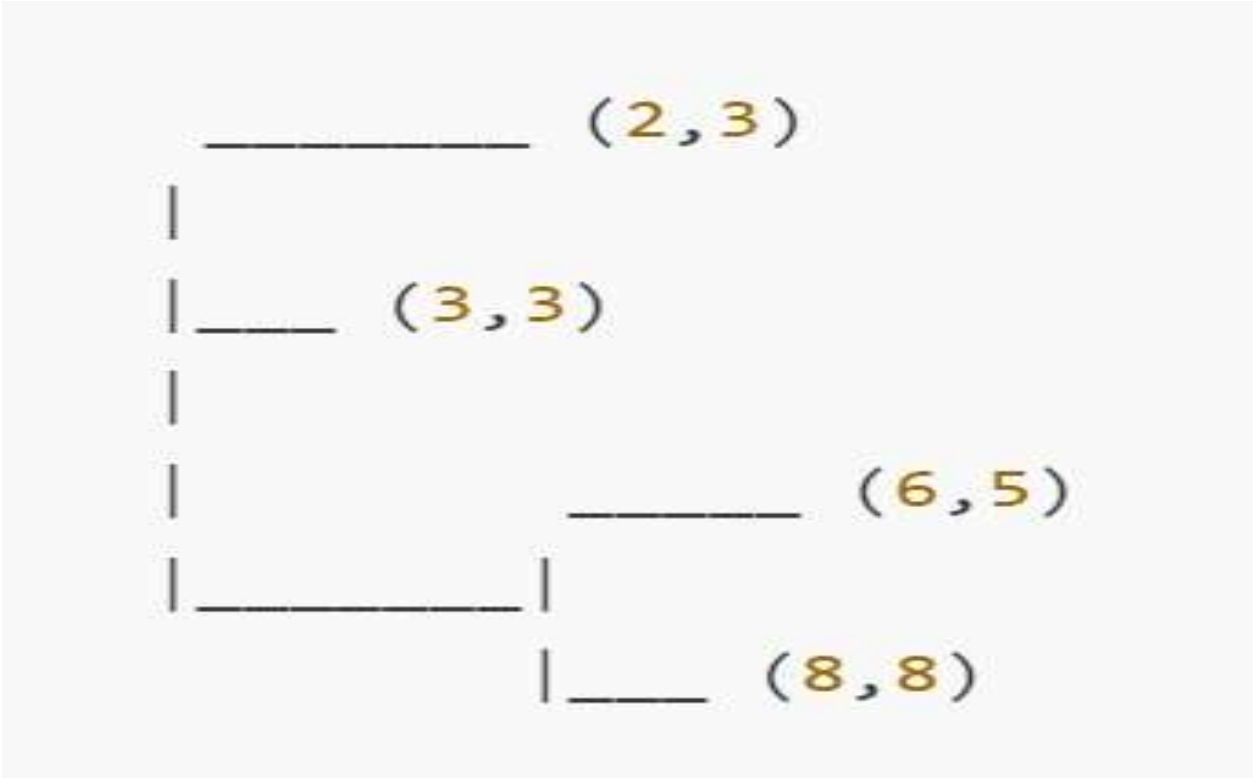
$$d((2,3),(3,3)) = 1, \quad d((2,3),(6,5)) = 5, \quad d((6,5),(8,8)) = 3.605$$

- **First Merge**:

The closest clusters are $\{(2,3)\}$ and $\{(3,3)\}$, so merge them:
$\{(2,3),(3,3)\},\{(6,5)\},\{(8,8)\}$.

4. **Update Distances**:
    - Recalculate the distances between the new clusters. Using **single linkage** (nearest neighbor), the distance between the cluster $\{(2,3),(3,3)\}$ and $\{(6,5)\}$ would be $d((3,3),(6,5)) = 3.605$.
5. **Second Merge**:
    - The next closest clusters are $\{(6,5)\}$ and $\{(8,8)\}$, so merge them: $\{(2,3),(3,3)\},\{(6,5),(8,8)\}$.
6. **Final Merge**:
    - The last merge is between $\{(2,3),(3,3)\}$ and $\{(6,5),(8,8)\}$.

Dendrogram Example:



Advantages:

- **No need to predefine the number of clusters**.
- **Dendrogram provides insight** into the hierarchical structure of the data, allowing flexible selection of the number of clusters.
- **Works well for small datasets** and visualizes the clustering process clearly.

Disadvantages:

- **Computationally expensive** for large datasets, especially divisive methods.
- **Sensitive to noise and outliers**, which can distort the dendrogram.
- **Cluster merging depends on linkage criteria**, which may yield different results based on the chosen method (single, complete, or average linkage).

Density based methods-DBSCAN

- DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is a popular density-based clustering algorithm in data mining.
- Unlike other clustering algorithms like K-means, which rely on predefined cluster shapes,
- DBSCAN identifies clusters based on the density of data points, making it effective at finding clusters of arbitrary shapes and dealing with noise.

Parameters:

Epsilon (ε): Defines the maximum distance between two points for one to be considered part of the other's neighborhood.

MinPts: The minimum number of points required to form a dense region.

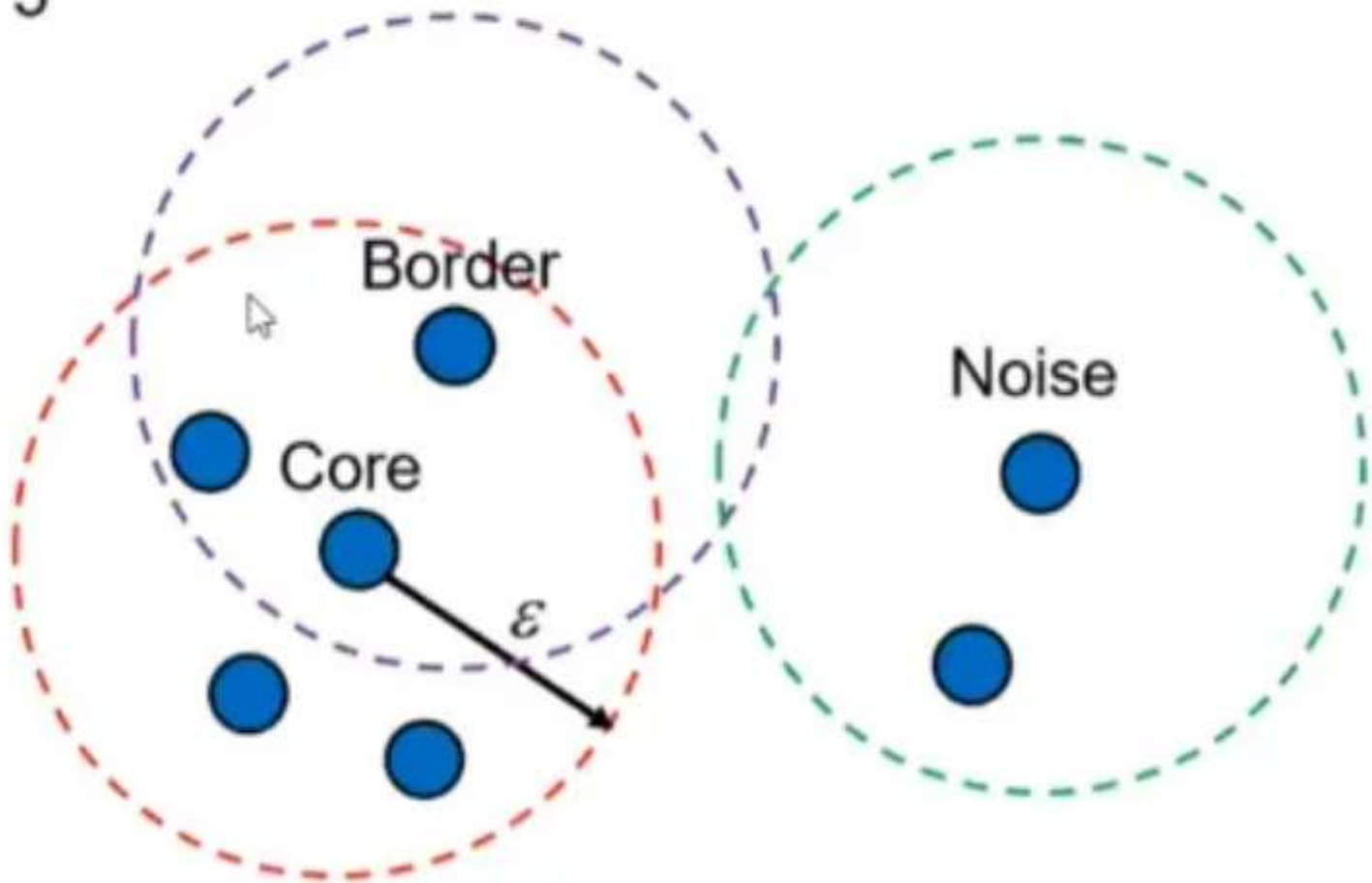
Core, Border, and Noise Points:

Core Point: A point with at least MinPts points within a radius of ε.

Border Point: A point within the neighborhood of a core point but doesn't have enough nearby points to be a core point.

Noise Point: Any point that is neither a core nor a border point, meaning it doesn't belong to any cluster.

MinPts = 5



Steps of DBSCAN

- **Start with an unvisited point** and check if it has enough neighbors within ε to be a core point.
- **Expand clusters** from core points by grouping them with all reachable points.
- **Repeat** for all points, marking those that don’t fit into clusters as noise

Advantages of DBSCAN

- **Detects arbitrary-shaped clusters**: DBSCAN is good for identifying clusters of various shapes, unlike K-means which only forms spherical clusters.
- **Robust to outliers**: It can classify points as noise, which makes it less sensitive to outliers.

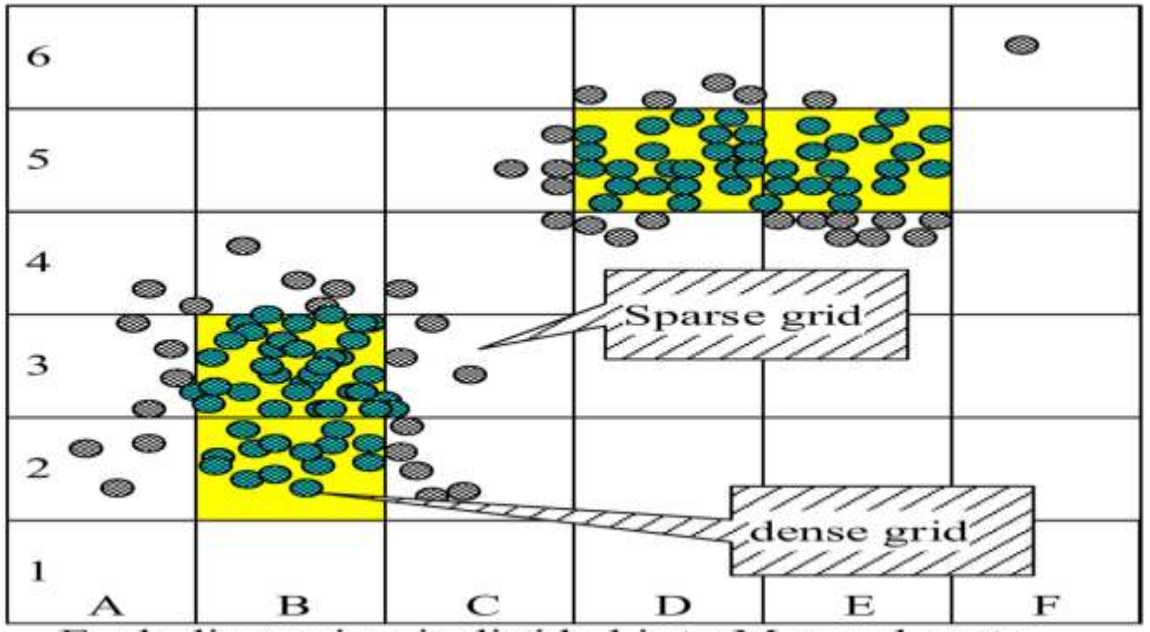
Disadvantages of DBSCAN

- **Difficulty with varying densities**: DBSCAN struggles when clusters have varying densities, as setting a single ε may not work well across different clusters.
- **Parameter sensitivity**: Choosing optimal values for ε and MinPts can be challenging, especially without prior knowledge of the data.

```
import numpy as np
from sklearn.cluster import DBSCAN
data = np.array([
    [1, 2],
    [2, 2],
    [2, 3],
    [8, 7],
    [8, 8],
    [25, 80]
])
db = DBSCAN(eps=3, min_samples=2)
clusters = db.fit_predict(data)
print("Cluster Labels:")
print(clusters)
```

Grid-Based clustering :

- The data space is divided into a **finite number of grid cells**
- Clustering is performed on the grid structure
- Instead of working on individual data points, it works on **cells**
- Processing time depends on number of grid cells,
- This makes it very efficient for **large datasets**.



- **Step 1: Divide Data Space into Grids**
- Suppose we have 2D data.
- We divide the space into equal-sized rectangles (cells).
- **Step 2: Calculate Density of Each Cell**
- Density = Number of data points inside the cell
- **Step 3: Identify Dense Cells**
- If density > threshold → Dense cell
  If density < threshold → Sparse cell (Noise)
- **Step 4: Merge Neighboring Dense Cells**
- Connected dense cells form clusters.

Mathematical Idea

- Total number of cells = m
- Number of points in cell i = Ni
- Threshold = T
- If Ni ≥ T → Cell is dense
  If Ni < T → Cell is sparse
- Time Complexity:
- O(m)
- Where m = number of grid cells

Important Grid-Based Algorithm

- **STING (Statistical Information Grid)**
- Statistical Information Grid

Working:

- Divides spatial area into hierarchical rectangular cells
- Each cell stores statistical information:
  - Count
  - Mean
  - Variance
  - Min
  - Max
- Works in a top-down manner.
- **Advantages:**
- Very fast
- Good for large spatial databases

Example (Step-by-Step)

- Suppose we have 2D points:
- (1,2), (2,2), (2,3), (8,7), (8,8), (25,80)
- Divide space into grid of size 5×5.
- Count points in each cell:

Cell	Points	Dense?
C1	3	Yes
C2	2	Yes
C3	1	No

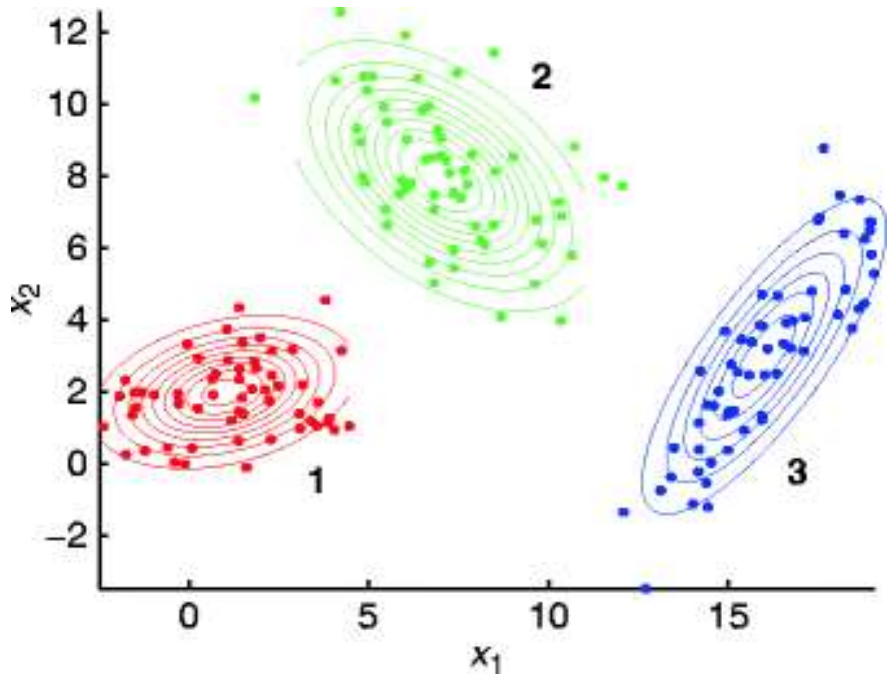
If threshold = 2

- Cluster 1 → Points in C1
  Cluster 2 → Points in C2
  Noise → C3

Model-Based Clustering :

Data is generated from a **mixture of probability distributions**

- Each cluster corresponds to a **statistical model**
- Instead of using only distance (like K-Means), it uses:
- Probability theory
- Statistical estimation



- **Why Model-Based is Needed?**
- K-Means assumes:
- Clusters are spherical
- Equal size

But real-world data:

- May be elliptical
- May have different sizes
- May overlap
- Model-based clustering handles this better.

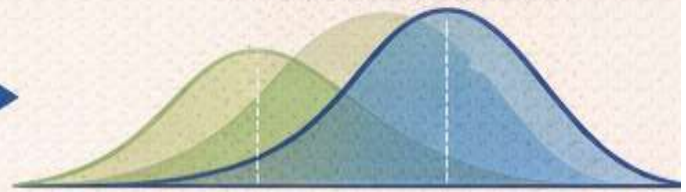
Basic Concept of Model-Based Clustering • Gaussian Mixture Model (GMM)

Suppose data comes from multiple populations.

Heights of Children
Normal Distribution

Hormal Distribution

Heights of Adults
Normal Distribution



Mixture of Normal Distributions

Mathematically:

$$P(x)=\sum_{k=1} \pi_k \cdot f_k(\theta_k)$$

$$P(x)=\sum_{k=1}^{K} \pi_k \bullet f_k(x \mid \theta_k)$$

- **K** = number of clusters
- π_k = Mixing probability (weight of cluster)
- f_k = Probability density function
- θ_k = Parameters (mean, variance)

Mixture of Normal Distributions

Basic Concept

- Suppose data comes from multiple populations.
- Example:
- Heights of children
- Heights of adults
- Each group follows a **normal distribution**.
- So total data = mixture of normal distributions.
- Mathematically:

$$P(x) = \sum_{k=1}^{K} \pi_k \cdot f_k(x|\theta_k)$$

Where:

- K = number of clusters
- π_k = mixing probability (weight of cluster)
- f_k = probability density function
- θ_k = parameters (mean, variance)

Clustering High-Dimensional

High-dimensional data means data with **many attributes (features)**.

- **For example:**
- Customer data with 200 features
- Gene expression data with 10,000+ attributes
- Text data where each word is a feature
- When the number of dimensions (features) increases, clustering becomes difficult.

Problems in High-Dimensional Clustering

1.Curse of Dimensionality

- As dimensions increase:
- Distance between points becomes almost the same
- Data becomes sparse
- Traditional algorithms like **K-Means** perform poorly

2. Irrelevant Features

- Not all attributes are useful for clustering.

3.Computational Complexity

- More dimensions → More calculations → More time

Approaches for Clustering High-Dimensional Data

1.Dimensionality Reduction Methods

Reduce number of features before clustering.

✓ **Feature Selection**

Select only important attributes.

✓ **Feature Extraction**

Create new reduced features.

Popular Techniques:

Principal Component Analysis (PCA)

Linear Discriminant Analysis (LDA)

t-Distributed Stochastic Neighbor Embedding (t-SNE)

- **Subspace Clustering**
- Clusters may exist only in **subset of dimensions**.
- Instead of using all features, find clusters in specific subspaces.
- **Example:**
- Customers may be similar only in income and spending score (not age)
- Popular Algorithms:
- **CLIQUE**
- **PROCLUS**
- **SUBCLU**

Constraint-Based Cluster Analysis

- **Constraint-Based Cluster Analysis** is a clustering approach where **user-defined constraints** are applied to guide the clustering process.
- Instead of only grouping based on similarity, we also consider **specific rules or conditions**.

Why Do We Need Constraints?

- Normal clustering (like K-Means) only uses distance.
- But in real life:
- We may want clusters within a certain **region**
- Or clusters with minimum population
- Or specific business rules
- So we add **constraints** to control clustering results.

Types of Constraints

Instance-Level Constraints

- These specify relationships between data points.
- ✓ **Must-Link Constraint**
- Two objects must be in the same cluster.
- Example:
  Student A and Student B must be grouped together.
- ✓ **Cannot-Link Constraint**
- Two objects cannot be in the same cluster.
- Example:
  Two competing companies should not be in the same cluster.

Cluster-Level Constraints

- These control properties of clusters.
- Minimum / Maximum size
- Diameter constraint (distance limit)
- Density constraint
- Shape constraint
- Example:
  Each cluster must contain at least 10 customers.

- **What is an Outlier?**
- An **outlier** is a data object that is **very different from other objects** in the dataset.
- It does not follow the normal pattern or behavior.
- Example:
- In a class where marks range from 40–90, a student scoring 5 is an outlier.
- In credit card transactions, a ₹5,00,000 purchase may be an outlier.

- **Types of Outliers**

1.Global Outlier

- Different from all other data points.
- Example:
  Very high salary in a company.

2.Contextual Outlier

- Abnormal in a specific context.
- Example:
  High temperature in winter.

- **Collective Outlier**
- A group of data points that together behave abnormally.
- Example:
  Sudden unusual network traffic pattern.
- **Methods of Outlier Detection**

Statistical Methods

- Assume data follows a distribution.
- Z-Score Method
- Box Plot (IQR method)

Distance-Based Methods

- Objects far away from others are outliers.
- **K-nearest neighbors algorithm**
- If distance is large → Outlier.

Clustering-Based Methods

- Small or distant clusters indicate outliers.
- **DBSCAN**
  - Points not belonging to any cluster → Outliers